RELATÓRIO DE AULA PRÁTICA

NOME DO ALUNO: Felipe Oliveira de Moraes
NOME DA DISCIPLINA: LINGUAGEM ORIENTADA A OBJETOS
UNIDADE 3: CLASSES E INTERFACES
AULA 4: TRATAMENTO DE EXCEÇÕES
TUTOR: Eduardo Viana de Almeida

OBJETIVOS:
Esta aula prática tem como objetivo permitir ao aluno aplicar, de forma integrada, os conceitos de classes abstratas e tratamento de exceções em Java. Por meio da criação de uma estrutura orientada a objetos para operações matemáticas, o aluno irá:

- Implementar herança com uso de classe abstrata e métodos abstratos;
- Criar subclasses concretas com comportamentos específicos;
- Desenvolver uma exceção personalizada;
- Utilizar blocos try-catch para o tratamento seguro de erros em tempo de execução;

ATIVIDADE PROPOSTA:
CÁLCULO DE OPERAÇÕES COM CLASSE ABSTRATA E TRATAMENTO DE EXCEÇÕES

Desenvolver um programa em Java que simule operações matemáticas utilizando classe abstrata e tratamento de exceções.

O aluno deverá:

1. Criar uma classe abstrata chamada OperacaoMatematica, com um método abstrato double calcular(double a, double b).

2. Criar duas classes concretas que herdam de OperacaoMatematica:
   - Soma: implementa o método para somar dois valores.
   - Divisao: implementa o método para dividir dois valores, mas lança uma exceção personalizada caso o segundo valor (b) seja zero.

3. Criar uma exceção personalizada chamada DivisaoPorZeroException, estendendo Exception.

4. Criar a classe Principal com o método main, onde:
   - Serão instanciadas as classes Soma e Divisao.
   - Serão feitas chamadas ao método calcular().
   - Será usado um bloco try-catch para tratar a exceção da divisão por zero.

O foco da atividade é aplicar abstração, herança e tratamento de exceções personalizadas de forma prática.

SOLUÇÃO DIGITAL:

Sublime Text: Editor de texto leve utilizado para a escrita do código-fonte.

JDK (Java Development Kit): Conjunto de ferramentas utilitárias para compilação e execução de sistemas Java.

Prompt de Comando (CMD): Interface de linha de comando do Windows utilizada para interagir com o compilador Java.

.

Downloads:

https://www.sublimetext.com/download

https://jdk.java.net/java-se-ri/22

DESENVOLVIMENTO:

O projeto foi desenvolvido utilizando a linguagem Java, focando na aplicação de conceitos avançados de POO e robustez de software.

Estrutura de Herança: Foi definida uma superclasse abstrata OperacaoMatematica que estabelece o contrato para as operações através do método calcular. As classes Soma e Divisao herdam essa estrutura, garantindo consistência no polimorfismo.

Exceção Personalizada: Criou-se a classe DivisaoPorZeroException estendendo Exception. Ela foi configurada com um construtor que utiliza super() para definir uma mensagem de erro padronizada, seguindo as convenções de encapsulamento.

Segurança de Tipos (Checked Exceptions): O método calcular na classe abstrata e em suas implementações declara explicitamente que pode lançar a exceção personalizada, o que obriga o tratamento via bloco try-catch na classe executável.

RESULTADO:

A execução no terminal (**CMD**) validou o comportamento esperado para diferentes cenários de entrada:

Operações Válidas: O programa exibiu corretamente os resultados da soma (10 + 5 = 15.0) e da primeira divisão (10 / 2 = 5.0).

Tratamento de Erro: Ao processar a divisão por zero, o fluxo foi desviado para o bloco catch, que imprimiu a mensagem: *"Erro: Não é possível dividir por zero."*.

Persistência do Fluxo: Graças ao bloco finally, a mensagem *"Fim do programa."* foi exibida mesmo após a ocorrência da exceção, confirmando que o sistema encerrou suas atividades de forma controlada.

C:\Trabalhos Faculdade\unidade 3\OperacaoMatematica.java - Sublime Text (UNREGISTERED)

```java
public abstract class OperacaoMatematica{
    public abstract double calcular(double a, double b) throws DivisaoPorZeroException;
}
```

C:\Trabalhos Faculdade\unidade 3\Divisao.java - Sublime Text (UNREGISTERED)

```java
public class Divisao extends OperacaoMatematica{
    @Override
    public double calcular(double a, double b)throws DivisaoPorZeroException{
        if(b == 0){
            throw new DivisaoPorZeroException();
        }
        return a / b;
    }
}
```

C:\Trabalhos Faculdade\unidade 3\Soma.java - Sublime Text (UNREGISTERED)

```java
public class Soma extends OperacaoMatematica{
    @Override
    public double calcular(double a, double b)throws DivisaoPorZeroException{
        return a + b;
    }
}
```

C:\Trabalhos Faculdade\unidade 3\Principal.java - Sublime Text (UNREGISTERED)

File Edit Selection Find View Goto Tools Project Preferences Help

OperacaoMatematica.java | Divisao.java | Soma.java | Principal.java | DivisaoPorZeroException.java

```java
public class Principal{
    public static void main(String[] args){
        OperacaoMatematica soma = new Soma();
        OperacaoMatematica divisao = new Divisao();
        try{
            System.out.println("Soma: " + soma.calcular(10, 5));
            System.out.println("Divisao correta: " + divisao.calcular(10, 2));
            System.out.println("Divisao com zero: " + divisao.calcular(10, 0));
        }
        catch(DivisaoPorZeroException e) {
            System.err.println(e.getMessage());
        }
        finally{
            System.out.println("Fim do programa.");
        }

    }
}
```

C:\Trabalhos Faculdade\unidade 3\DivisaoPorZeroException.java - Sublime Text (UNREGISTERED)

File Edit Selection Find View Goto Tools Project Preferences Help

OperacaoMatematica.java | Divisao.java | Soma.java | Principal.java | DivisaoPorZeroException.java

```java
public class DivisaoPorZeroException extends Exception{
    public DivisaoPorZeroException(){
        super("Erro: Não é possível dividir por zero.");
    }
}
```

C:\WINDOWS\system32\cmd.

```
C:\Trabalhos Faculdade\unidade 3>javac Principal.java

C:\Trabalhos Faculdade\unidade 3>java Principal
Soma: 15.0
Divisao correta: 5.0
Erro: Não é possível dividir por zero.
Fim do programa.

C:\Trabalhos Faculdade\unidade 3>
```

CONCLUSÃO:

A atividade demonstrou que a arquitetura de um software deve prever não apenas o "caminho feliz", mas também o gerenciamento de falhas. A implementação reforçou que:

- O uso de **modificadores de acesso public** em arquivos separados é essencial para a organização e visibilidade do projeto em ambiente profissional.
- O **polimorfismo** permite tratar diferentes operações de forma genérica, facilitando a manutenção e a adição de novas funcionalidades (como subtração ou multiplicação) no futuro.
- O **tratamento de exceções personalizadas** eleva a qualidade do software ao fornecer feedbacks claros e evitar que erros matemáticos causem o travamento do sistema operacional.